

AI Guardrail — Agent Governance Layer: Design & Module Plan

Status: §7.1 (the AI Guardrail side) is implemented and tested — 116 tests passing, verified live end-to-end against the real DPDP Engine. §7.2/§9 (the VOXA/Kotlin side) is not built by this project; this document is VOXA's integration reference. §4.1 reflects the exact contract as implemented, not a proposal.

Companion inputs this design is grounded in:

- Internal call transcript, Mahesh Ramakrishnan / Mausam Ghosh, 2026-09-20 (agent identity approaches, the agent-metadata vs. activity-log split, registration options).
- `GOVERNANCE_LAYER_MOVEMENT_PLAN.md` (`D:\PoC_Workspaces\AI_DPDP_Engine`) — the DPDP Engine's own explicit scope-split document, addressed directly to this project.
- Direct code investigation of `D:\PoC_Workspaces\AI-Governance-version2` (existing audit/alarm/auth building blocks) and `D:\PoC_Workspaces\JIO-VOXA-version-1` (the first real external agent this layer will serve).

1. Scope split (reaffirmed, unchanged from the DPDP Engine's own plan)

Concern	Lives in
Notices, consents, consent-lifecycle decisions, DPDP-specific compliance events	DPDP Engine — unchanged, pure consent manager, nothing agent-specific
Agent identity / registry, invoking-user tracking, general Activity Log, agent-governance compliance events, Stats	AI Guardrail — new Agent Governance Layer (this document)

The "Decision Passthrough" module below is **not** a second consent engine. It is an identity-verified front door in front of the existing, unmodified `dpdp_client.check_decision(...)` call. The DPDP Engine remains the sole authority on "is there consent for this" — this layer never re-derives that answer.

2. Data model

2.1 Agent Registry (static — one row per registered agent, written once at registration)

Field	Notes
<code>agent_id</code>	Server-generated UUID. The technical identity.
<code>agent_name</code>	Human-readable label, e.g. "VOXA — Swiggy Skill".
<code>agent_secret_hash</code>	Hashed at rest, never stored or logged in plaintext.
<code>business_unit</code>	Org unit that owns this agent.

<code>owner_name</code>	The person/team accountable for this agent — distinct from <code>business_unit</code> , this is a real contact, not a label.
<code>location_of_deployment</code>	e.g. <code>on-device:android</code> , or a server region/hostname for a cloud-deployed agent.
<code>in_house_or_external</code>	Flag.
<code>identity_assignment_timestamp</code>	Set once, at registration.
<code>status</code>	<code>active</code> / <code>revoked</code> — minimal lifecycle only; the fuller life-cycle enum discussed on the call is explicitly deferred, not needed for this POC.

Deliberately excluded from this table: any per-user field. See 2.2 for why.

2.2 Per-call signature → Activity Log row (dynamic — one row per call, append-only)

```
{
  "ts": "2026-09-20T10:00:00Z",
  "agent_id": "...",
  "invoking_user_id": "...",
  "action": "...",
  "outcome": "SERVED | BLOCKED",
  "latency_ms": 0,
  "correlation_id": "..."
}
```

`invoking_user_id` lives here, not in the registry, because one agent can be used by many different people over time (or, for a single-user app like VOXA, by whatever stable pseudonym represents that one account/device — see §6). Baking a single user identity into the static registry would be wrong by construction the moment an agent serves more than one person.

No raw request/response content is ever logged here — `action` is a short operation label (e.g. "swiggy_cart_update"), not the user's message. See §5 for the reasoning.

External validation of this split: this mirrors Alexa Skills Kit's own architecture — a static, per-skill **Skill ID** assigned once at registration, versus a separate **User ID** (and Device ID) carried on every individual request, deliberately scoped so the same identity fields never get collapsed into one. Amazon's own design treats "which skill" and "who's talking to it right now" as two different identifiers for exactly this reason.

2.3 Governance Compliance Events (agent-scoped, append-only)

Same *shape* as the DPDP Engine's own `compliance_events` table (event type, version, severity, subject, structured categories, source, reason code, correlation id, two timestamps) but the subject is an `agent_id` , not a DPDP `subject_ref` . Suggested event types, parallel to DPDP's own vocabulary: `AGENT_IDENTITY_UNVERIFIED` , `AGENT_UNAUTHORIZED_ACTION` , `AGENT_POLICY_VIOLATION` , `AGENT_SCOPE_EXCEEDED` . Raised automatically by the Identity Verification and Decision Passthrough modules on failure — never sent by the calling app itself.

2.4 Stats

No new store. Pure read-side aggregation over 2.1–2.3 (calls by outcome, calls by agent, events by severity, distinct agents/users seen) — same approach as the DPDP Engine's own `GET /v1/stats` (a `GROUP BY` query, no running counters maintained separately).

3. Identity issuance & verification flow

1. **Registration** (once per install/deployment): `POST /v1/agent/register` — self-registration (transcript's Option 3: as good as CI/CD-driven registration, avoids the "owner forgets to wire in the ID" weakness of manual registration and the ID-proliferation risk of auto-assign-on-first-call). Request carries `agent_name`, `business_unit`, `owner_name`, `location_of_deployment`, `in_house_or_external`. Response: `agent_id`, a freshly generated **agent-specific** secret (not a shared token — a shared secret is explicitly called out, in the DPDP Engine's own movement plan, as "the anti-pattern for agent identity: one shared secret can't tell agents apart"), and `identity_assignment_timestamp`.
2. **Every subsequent call** carries `X-Agent-Id` + the issued secret. Verification happens **inline**, as part of the same request (not a separate pre-flight "verify identity" round trip) — unknown `agent_id`, wrong secret, or `status=revoked` → reject immediately with **401** (see §4.1 for the exact error shape), before the request reaches the decision or logging logic at all. Only a verified identity's request is processed.
3. **Open item, not resolved here:** reinstall/re-registration behavior. Either accept some identity churn (a reinstall becomes a new `agent_id`), or design an idempotent registration key — needs an explicit decision, not an assumption.

4. New API surface (AI Guardrail's own, distinct from anything DPDP-labeled)

Route	Purpose
<code>POST /v1/agent/register</code>	One-time registration (§3).
<code>POST /v1/agent/decisions/check</code>	Identity-verified passthrough to <code>dpdp_client.check_decision(...)</code> ; logs the outcome to the Activity Log; raises a Governance Compliance Event on identity failure or a DPDP <code>DENY / UNKNOWN</code> .
<code>POST /v1/agent/activity</code>	Lightweight, non-gating activity log entry for calls that don't need a consent decision at all (e.g. a read operation).
<code>GET /v1/agent/*</code> (admin)	List/browse endpoints backing the four new Admin UI tabs.

Deliberately namespaced under `/v1/agent/...`, distinct from the DPDP Engine's own `/v1/decisions/check` etc., so nobody integrating against either service confuses the two contracts.

4.1 Exact wire contract (as implemented — `api.py`)

Auth headers, every route except `/register`: `X-Agent-Id: <agent_id>` + `Authorization: Bearer <secret>`. Not a body field.

`POST /v1/agent/register` — no auth required (this is how it's obtained).
Request: `{agent_name, business_unit, owner_name, location_of_deployment, in_house_or_external}`.

Response (200): {"agent_id": "...", "agent_secret": "...", "identity_assignment_timestamp": "..."} .

The secret's JSON key is `agent_secret` , shown exactly once, here.

POST /v1/agent/decisions/check .

Request: {subject_ref, purpose, operation, data_categories, recipient_ref?, policy_context?, correlation_id?} .

Response (200): `dpdp_client.check_decision(...)` 's return value plus an echoed `correlation_id` : {"decision", "guard_failed", "error", "decision_id", "notice_version", "reason_code", "correlation_id"} .

`correlation_id` is optional in the request — **if omitted, the server generates one and echoes it back** (not left to `dpdp_client.py` 's own internal fallback, so the same id ties together the DPDP Engine call, this layer's activity/event rows, and the response the caller sees).

This endpoint has **no `latency_ms` field** — only `/activity` captures that.

POST /v1/agent/activity (non-gating).

Request: {invoking_user_id, action, outcome, latency_ms?, correlation_id?} .

Response (200): {"status": "logged", "correlation_id": "..."} (same omit-then-generate-and-echo rule).

`ts` is always server-computed on receipt — there is no client-timestamp field. `latency_ms` is optional and entirely client-supplied and unvalidated; the Guardrail does not compute it, so the timer start point is the calling agent's own choice to define.

Error shape on identity failure: always **401** (403 is not used), FastAPI's default `HTTPException` body: {"detail": "Agent identity could not be verified."} . Unknown `agent_id` , wrong secret, and a revoked agent all produce this **same** generic message — `verify_agent()` returns `None` uniformly for all three failure modes by design, so there is currently no machine-readable way to tell them apart. This is a deliberate choice, not an oversight: not confirming whether a given `agent_id` exists at all is a reasonable security posture (comparable to a login form that doesn't confirm whether a username exists). Revisit only if a concrete need for finer-grained client-side error handling shows up.

A genuine network failure (Guardrail unreachable/timeout) never produces this shape at all — no HTTP response comes back — so a caller can already distinguish "identity rejected" (got a 401 with this body) from "couldn't reach the Guardrail" (no response) without any additional signal needed.

5. Activity logging content policy

No raw request/response content, ever. This isn't a new rule invented for this layer — it's the same discipline already established everywhere in this project: `audit_logger.py` never logs raw input (only masked text or a hash), the Guardrail Activity panel deliberately never shows message content, and the DPDP Engine's own compliance events outright reject `raw_prompt` / `message` / `text` fields. If proof of *what* happened is ever needed without storing it, use a `content_hash` (SHA-256), the same pattern the DPDP Engine already uses. `action` is a short operation label, not a message body.

6. Ingress-validation principle (revised — see rationale below)

Not every message gets validated externally. Routing every keyed-in/spoken utterance through a network call to this Guardrail *before* an on-device LLM even processes it would defeat the actual purpose of running the LLM on-device: it would make the assistant's basic operation depend on network + Guardrail uptime, add a round-trip's latency to every turn, and — the sharper point — ship 100% of raw conversational content off the device by default, which is a *worse* privacy posture than keeping it on-device, not a better one.

The corrected rule: validate only at the point of external effect. A network call to this layer happens only when an agent is about to take an action that mutates state outside the device (a cart write, a payment) — never merely because the user said something. And even then, what crosses the network is operation *metadata* (agent id, purpose, operation, data category, correlation id) — never the raw utterance or tool arguments, per §5.

This also matches why the classic "injection guard protects the platform" justification is weaker for a single-user, on-device agent than for the original AI-Governance chatbot: that system's LLM has `tool_broker` access to *other customers'* records, so a hijacked prompt could exfiltrate someone else's data. An on-device assistant like VOXA only ever acts within one user's own linked accounts — there's no other tenant's data to protect by screening every utterance over the network.

7. Module-wise development plan

7.1 AI Guardrail side (built and tested first)

Module	Responsibility
Agent Registry	Schema (§2.1) + <code>POST /v1/agent/register</code> .
Identity Verification	Inline dependency/middleware checked on every <code>/v1/agent/*</code> call before anything else runs.
Decision Passthrough	Identity-verified wrapper around the existing <code>dmdp_client.check_decision(...)</code> — no changes to <code>dmdp_client.py</code> itself. New purposes (e.g. <code>voxa_swiggy_cart_management</code> , <code>voxa_swiggy_payment_initiation</code>) get registered on the DPDP Engine side, the same way <code>device_diagnosis_ai_analysis</code> was for JioCare Helper — a DPDP-Engine-side task, not this layer's.
Activity Log	Append-only store; written by the Decision Passthrough module (every check, any outcome) and by the lighter <code>POST /v1/agent/activity</code> for non-gated reads.
Governance Compliance Events	Raised automatically on identity failure or DPDP denial — never sent directly by a calling app.
Stats	Pure aggregation, no new writes (§2.4).
Admin UI	One new top-level button in <code>App.jsx</code> (mirrors the existing flat nav: Chat Bot / Admin Configuration / Analytics / Testing Dashboard), opening a new page with its own 4-tab bar (Registered Agents / Activity Log / Compliance Events / Stats), following <code>AdminConfig.jsx</code> 's existing tab+table+Refresh pattern exactly.

7.2 VOXA side (spec to hand off once 7.1 is built and tested — not built by this project)

Module	Responsibility
<code>AiGuardrailClient.kt</code>	New client, same shape as the existing <code>SwiggyMcpClient</code> / <code>KiteClient</code> .
Registration at install	In <code>MainActivity.onCreate</code> , alongside where <code>swiggyAuth</code> / <code>kiteAuth</code> are already constructed — register once, persist <code>agent_id</code> + secret via <code>EncryptedSharedPreferences</code> , reusing the exact storage pattern already proven for <code>swiggy_auth</code> .

Subject pseudonym	New state VOXA doesn't have today — confirmed by direct investigation, VOXA has no end-user identity of any kind. Generate a stable per-install/per-account UUID once, persist it the same way, use it as <code>subject_ref</code> for DPDP consent checks.
Write-operation gating	Inside <code>SwiggyMcpClient.callTool()</code> : for <code>update_food_cart</code> (confirmed existing) and a future payment-initiation tool (not yet present in the app), call <code>POST /v1/agent/decisions/check</code> before the real Swiggy MCP call; block on anything but <code>ALLOW</code> . Read operations pass straight through, optionally with a non-gating <code>POST /v1/agent/activity</code> call.
Activity logging hook	The existing <code>onToolExecuted</code> callback is already the right attachment point — no restructuring needed.

8. Worked example: VOXA / Swiggy (grounded in direct code investigation, not assumption)

Confirmed scope for this VOXA integration: Swiggy and Yahoo Finance only. Kite and Teams are present in the VOXA codebase but explicitly excluded from this plan.

Confirmed from `D:\PoC_Workspaces\JIO-VOXA-version-1 :`

- On-device Gemma via LiteRT-LM, genuine LLM tool-calling (not a sentinel-token protocol), one shared `Conversation /tool` registry across Swiggy/Kite/Yahoo Finance/Teams — no per-skill sandboxing at the model level.
- Swiggy calls go through Swiggy's own MCP server (`https://mcp.swiggy.com/food` , OAuth 2.1 + PKCE) via a single choke point, `SwiggyMcpClient.callTool()` .
- Reads: `get_addresses` , `search_restaurants` , `get_restaurant_menu` , `get_food_cart` . Write: `update_food_cart` only — **no checkout/payment call exists in the app today**; a Settings-screen disclaimer explicitly tells the user to check out in the real Swiggy app.
- No phone number, no payment/card data, no GPS anywhere in the Swiggy code path (the manifest doesn't even request location permission).
- Zero existing consent/PII/toxicity/injection logic anywhere in the codebase — full-tree grep, zero hits. This is genuinely new construction on VOXA's side, not an extension of a partial system.

Recommended consent surface for Swiggy: gate only `update_food_cart` (and, once it exists, a payment-initiation tool) — not the reads, since none of them touch anything beyond commercial/public data or an address tag. Suggested purpose/operation/category, reusing already-established DPDP vocabulary rather than inventing new names: `purpose=voxa_swiggy_cart_management` , `operation=UPDATE_CART` , `data_categories=["ORDER_DATA"]` ; for a future payment tool, `purpose=voxa_swiggy_payment_initiation` , `operation=INITIATE_PAYMENT` , `data_categories=["PAYMENT_TOKEN"]` — the same operation/category names already proven working in this project's own payment-gate integration.

9. Per-skill hand-off checklist for VOXA (Swiggy + Yahoo Finance)

Each VOXA skill gets its **own** Agent Registry entry and its own `agent_id /secret` — per the Alexa Skills Kit precedent (§2.2): every skill is a separately identified thing in that model too, regardless of whether it happens to process personal data. What differs sharply between these two skills is *what gets sent to the Guardrail and when* — one has a real DPDP consent surface, the other has none at all, confirmed by direct code investigation, not assumption.

9.1 Skill profile comparison

	Swiggy	Yahoo Finance
Personal/user data involved	Yes — address tag, cart contents, order data	None — public market data only, no account, no credentials (<code>YahooFinanceClient.kt</code> 's own doc comment: "no account, no credentials involved anywhere here")
Write operations	Yes — <code>update_food_cart</code> (+ a future payment tool)	No — <code>search</code> and <code>get_quote</code> only, both read-only
Needs a DPDP consent-check call	Yes , on the write path only	No — there is no Data Principal data being processed, so there is no purpose to register and nothing to gate
Needs an Agent Registry entry	Yes	Yes — for identity/traceability, not for consent
Needs Activity Log entries	Yes, every tool call	Yes, every tool call — pure observability (is the gateway up, how often is it called), not privacy-driven
Can raise a Governance Compliance Event	Yes — <code>AGENT_IDENTITY_UNVERIFIED</code> (bad credentials) or <code>AGENT_UNAUTHORIZED_ACTION</code> / <code>AGENT_POLICY_VIOLATION</code> (a real DPDP <code>DENY</code>)	Only <code>AGENT_IDENTITY_UNVERIFIED</code> — the data-related event types can never fire here, since no decision-check call exists for this skill at all

9.2 Registration (both skills, done once each, same mechanism, §3)

VOXA registers **two** separate agents at first run, not one for "VOXA" generally:

1. `agent_name="VOXA – Swiggy Skill"` , plus `business_unit` , `owner_name` , `location_of_deployment="on-device:android"` , `in_house_or_external` .
2. `agent_name="VOXA – Yahoo Finance Skill"` , same other fields.

Persist each returned `agent_id` + secret separately (e.g. two new `EncryptedSharedPreferences` keys, mirroring the existing `"swiggy_auth"` naming convention already used in the app — this is a pattern VOXA's own code already establishes, not something new being introduced).

One subject pseudonym, shared across both skills — not one per skill. This is a deliberate departure from the Alexa comparison, worth stating explicitly rather than silently copying the pattern: Alexa scopes `userId` differently per skill specifically to stop two *unrelated third-party developers* from correlating the same person across their independent skills. That threat doesn't exist here — both skills are the same app, same team, same device. The DPDP Engine already isolates consent per `(subject_ref, purpose)` , not per `subject_ref` alone (proven by the JioCare Helper integration: "a second, hypothetical purpose does not inherit this grant... `consent_records` is keyed on `(subject_ref, purpose)` ") — so reusing one pseudonym across skills doesn't leak anything between them; the purpose field already provides that isolation. One persisted UUID per install is simpler for VOXA to build than N, with no real privacy cost.

9.3 Swiggy — what goes, when

Reads (`search_restaurants` , `get_restaurant_menu` , `get_food_cart` , `get_addresses`):

- No consent-check call — there's no write/legal-basis question for a read.
- Optional, non-gating: `POST /v1/agent/activity` *after* the Swiggy call completes (never blocks it), with `agent_id =Swiggy`, `invoking_user_id =the shared pseudonym`, `action="swiggy_read:<tool_name>"` , `outcome=SERVED` .
- Never send: raw address text, menu contents, cart items, prices, the Swiggy OAuth token.

Writes (`update_food_cart` , and a future payment-initiation tool):

- `POST /v1/agent/decisions/check` **before** the real Swiggy MCP call, with `agent_id =Swiggy`, `subject_ref =the shared pseudonym`, `purpose="voxa_swiggy_cart_management"` , `operation="UPDATE_CART"` , `data_categories=["ORDER_DATA"]` (for a future payment tool: `purpose="voxa_swiggy_payment_initiation"` , `operation="INITIATE_PAYMENT"` , `data_categories=["PAYMENT_TOKEN"]`).
- Anything but `ALLOW` → block the Swiggy call entirely; do not call `SwiggyMcpClient.callTool()` for that write.
- `ALLOW` → proceed with the real Swiggy call, then log a `SERVED` activity entry.
- Never send: cart contents, item names/prices, the OAuth token, the raw user utterance.
- Exact insertion points: `SwiggyTools.kt` 's `addItemToCart` (the tool function that calls `update_food_cart`) and the auto-add-winner path inside `findBestRestaurantForDish` — both currently call `SwiggyMcpClient.callTool("update_food_cart", ...)` directly; the decision-check call needs to sit immediately before that call in both places.

9.4 Yahoo Finance — what goes, when

- Only one model-facing tool, `getStockPrice` , which itself calls two read-only MCP methods (`search` , `get_quote`) — confirmed no personal data anywhere in this path.
- **No consent-check call, ever** — there is no Data Principal data involved, so there is nothing to register a DPDP purpose for and nothing to gate. Do not invent a purpose for this just for consistency with Swiggy; that would be registering a consent requirement where none exists.
- Recommended, non-gating: `POST /v1/agent/activity` after each lookup, with `agent_id =Yahoo Finance`, `action="stock_price_lookup"` , `outcome=SERVED` (or `BLOCKED` -equivalent only on a network/timeout error) — purely operational monitoring (is the public gateway reachable), not privacy-driven.
- Never send: the actual company/ticker query text or the returned price data — not because either is sensitive (it isn't), but to keep one uniform "never send content, only a short action label" rule across every skill rather than carving out a content-is-fine-here special case.
- Exact insertion point: `YahooFinanceTools.kt` 's `getStockPrice` function, at its existing `onToolExecuted` callback (already called once on the error-path and once on the success-path) — that's already the natural, existing hook to attach the activity-log call to.

9.5 What travels on every call, summarized

Field	Scope	Notes
-------	-------	-------

X-Agent-Id + secret	Per skill	Different for Swiggy vs. Yahoo Finance calls
invoking_user_id / subject_ref	Per install (shared)	One pseudonym, reused across both skills — see §9.2
correlation_id	Per request	Fresh every call, never reused
Message/tool-argument content	Never sent	No skill's raw content crosses this boundary — metadata only, per §5

10. Open items requiring an explicit decision before implementation

1. Reinstall/re-registration behavior (§3.3) — accept identity churn, or build idempotent registration?
2. VOXA's subject-pseudonym mechanism (§7.2, §9.2) is genuinely new work on VOXA's side — needs their commitment, not just this layer's API contract.
3. Hard-block-unregistered-agents policy (transcript's Option 4a) is an organizational decision, not a technical default — out of scope for this design until explicitly requested.
4. Exact DPDP Engine purpose registration for the two Swiggy purposes above is a DPDP-Engine-side task to schedule separately. Yahoo Finance needs no such registration at all (§9.4).

11. Sequencing

Build and test 7.1 (AI Guardrail's own governance layer) first, end-to-end, against a stub/manual caller. Only once that contract is stable does VOXA build 7.2 against it — so both sides can verify they generate/consume the same agent_id /metadata shape from a working reference, not a moving target.